Pengantar Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) adalah teknik yang menggabungkan pencarian informasi dengan model bahasa besar (LLM). Sebelum LLM menjawab, sistem mencari potongan dokumen yang relevan dari basis pengetahuan, lalu potongan tersebut diberikan kepada LLM sebagai konteks.

Tujuan utama RAG adalah mengurangi halusinasi. LLM biasa hanya mengandalkan pengetahuan yang tersimpan dalam parameternya, sehingga dapat mengarang jawaban. Dengan RAG, jawaban dibatasi pada dokumen yang diambil, dan sumbernya dapat ditelusuri.

Pipeline RAG terdiri dari dua tahap besar. Tahap pertama adalah ingestion, yaitu membaca dokumen, memecahnya menjadi chunk, mengubah setiap chunk menjadi vektor embedding, dan menyimpannya di vector database. Tahap kedua adalah query, yaitu mengubah pertanyaan menjadi vektor, mencari chunk paling mirip, dan meminta LLM menyusun jawaban.

Keuntungan RAG dibanding fine-tuning adalah pengetahuan dapat diperbarui hanya dengan mengganti dokumen, tanpa melatih ulang model. Biaya dan waktu yang dibutuhkan juga jauh lebih kecil.

Chunking dan Embedding

Chunking adalah proses memecah dokumen panjang menjadi potongan kecil. Chunk size menentukan panjang maksimum setiap potongan. Chunk yang terlalu kecil kehilangan konteks, sedangkan chunk yang terlalu besar membuat makna pencarian menjadi kabur dan boros token.

Chunk overlap adalah bagian teks yang sengaja diulang di antara dua chunk bersebelahan. Overlap menjaga agar kalimat yang terpotong di batas chunk tetap utuh di salah satu chunk, sehingga informasi penting tidak hilang di perbatasan.

Embedding adalah representasi teks dalam bentuk vektor angka. Teks dengan makna serupa memiliki vektor yang berdekatan. Model paraphrase-multilingual-MiniLM-L12-v2 menghasilkan vektor berdimensi 384 dan mendukung banyak bahasa, termasuk bahasa Indonesia.

Model embedding memiliki batas panjang input. Untuk MiniLM-L12-v2 batas maksimumnya adalah 128 token, sehingga chunk yang lebih panjang dari itu akan terpotong saat di-embed.

Vector Database dan Parameter Generasi

Vector database menyimpan embedding dan mampu mencari vektor terdekat dengan cepat. Contohnya adalah Qdrant, Pinecone, dan Chroma. Kemiripan biasanya diukur dengan cosine similarity, dengan nilai mendekati 1 berarti sangat mirip.

Parameter TOP_K menentukan berapa banyak chunk teratas yang diambil dari vector database untuk dijadikan konteks bagi LLM. Nilai TOP_K yang kecil dapat melewatkan informasi, sedangkan nilai yang besar dapat memasukkan konteks yang tidak relevan.

Temperature adalah parameter generasi yang mengatur tingkat keacakan pemilihan token oleh LLM. Temperature rendah, misalnya 0.0, membuat jawaban lebih deterministik dan konsisten. Temperature tinggi, misalnya 0.8, membuat jawaban lebih bervariasi dan kreatif tetapi berisiko menyimpang dari fakta.

Temperature hanya memengaruhi tahap generasi. Proses retrieval tidak berubah karena pencarian vektor tidak melibatkan pengambilan sampel acak.